

LLM Production Deployment

Interview Study Guide — ML Infra / Platform Role

1. Core serving fundamentals

Prefill vs. decode

- Prefill: processing the input prompt in one pass. Compute-bound — parallelizes well across the whole prompt.
- Decode: generating output tokens one at a time, autoregressively. Memory-bandwidth-bound — each step re-reads the KV cache and model weights but does relatively little compute.
- Mixing both on the same GPU causes a classic failure mode: a large prefill job hogs compute and stalls decode latency for everyone else on that instance.

Interview angle: if asked "why not just round-robin load balance?" — this is the answer: Round robin has no concept of prefill vs decode or cache locality.

Disaggregated prefill/decode

- Fix for the above: run separate pools of GPU workers — one for prefill, one for decode.
- A request's prompt is processed on a prefill worker; the resulting KV cache is then transferred to a decode worker, which streams out tokens.
- Transfer is not free — moving KV cache across nodes costs bandwidth and adds latency. Worth it when throughput/latency gains outweigh transfer cost (large batches, big models); less clear-cut for small, low-traffic deployments.
- Production systems use direct high-speed transfer (RDMA/NVLink, e.g. NIXL in llm-d) for this hot path rather than routing through shared disk/storage, which is far higher latency under concurrency.

KV cache

- Stores the key/value tensors for every token already processed, so the model doesn't recompute attention over the whole prefix at each new token.
- Grows with batch size $\times$ sequence length — often the single biggest memory consumer at serving time, larger than you'd expect from weights alone.
- PagedAttention (vLLM's core innovation) manages KV cache in fixed-size blocks like OS virtual memory pages, eliminating memory fragmentation and enabling much higher batch sizes.
- Prefix / cache-aware routing: if a new request shares a prompt prefix with one recently served on instance A, route it back to A so it can reuse cached KV blocks instead of recomputing — this is what a "plain load balancer" can't do.
- Multi-tier caching (e.g. LMCache): KV blocks can live across GPU memory, CPU RAM, and local SSD/remote store, trading a bit of latency for much higher cache hit rates and reuse across many requests — different use case from the prefill→decode handoff.

2. The modern serving stack

vLLM

- High-throughput, memory-efficient inference engine. Core contribution: PagedAttention.
- Handles continuous batching (dynamically adding/removing requests from a running batch rather than static batching), quantization support, speculative decoding, and broad hardware/model support.

llm-d

- Kubernetes-native orchestration layer built on top of vLLM (and SGLang). Not a replacement for vLLM — think of it like Linux (vLLM) vs Kubernetes (llm-d).
- Three pillars: (1) an inference-aware scheduler/gateway that routes on real signals (queue depth, KV cache hits, load) instead of round robin, (2) disaggregated prefill/decode, (3) a multi-tier KV cache.
- Built by Red Hat, Google Cloud, IBM Research, NVIDIA, CoreWeave and others; now a CNCF sandbox project.

Inference Gateway (IGW)

- An official Kubernetes Gateway API extension for AI-aware routing — the component that actually implements smart, signal-based routing decisions.

3. Kubernetes production architecture

Request path

- Client request → Inference Gateway (auth, rate limiting, routing) → Prefill pool → KV cache handoff → Decode pool → streamed tokens back to client.
- Prefill and decode are separate Deployments/StatefulSets with independent scaling — this is the load-bearing architectural decision the rest of the system hangs off of.

Node & scheduling setup

- GPU nodes live in a dedicated node pool with taints/tolerations so only inference workloads land there.
- Model weights are not baked into container images (too large, slow rollouts) — pulled from object storage (S3/GCS) or a shared volume at startup, sometimes streamed layer by layer.

Autoscaling

- Not standard CPU-based HPA — driven by custom metrics: queue depth, time-to-first-token, KV cache utilization, active-sequence count.
- KEDA is the common tool for scaling on these custom/external metrics (via its Prometheus scaler, or an external queue length if one is in the pipeline).
- Prefill and decode pools often scale on different signals since they saturate differently — prefill on queue length, decode on active sequences / memory pressure.
- Spot/preemptible GPU capacity is a common cost lever for burst traffic, usually favored more for decode (checkpointable / migratable state) than for prefill.

Where is queue depth actually stored?

- Not in a database — it's a live, in-memory number inside each vLLM instance's scheduler, exposed as a Prometheus metric (e.g. `num_requests_waiting`).
- The gateway maintains its own view of per-backend queue depth (usually derived from the same scraped metrics) to make routing decisions.
- KEDA reads it via a PromQL query against Prometheus, or — if an external queue (Redis, Kafka) sits in front of the pool for admission control — directly as queue length / consumer lag.
- Most low-latency serving stacks keep the queue in-process and just export it as a metric, rather than routing every request through an external broker, because that adds latency to the hot path.

Security

- Network policies: only the gateway can reach inference pods directly — no direct external access to GPU pods.
- mTLS between gateway and pools (service mesh — Istio/Linkerd) so prompts and KV transfers aren't sniffable in-cluster.
- Secrets via K8s Secrets backed by an external vault (Vault, AWS Secrets Manager) — never plaintext in images/env vars.
- Pod Security Standards (restricted profile), image scanning in CI, RBAC scoped so only the deployment pipeline can touch inference workloads.
- Multi-tenancy: namespace isolation + resource quotas so one tenant/model can't starve GPU memory from another.

Observability

- Prometheus scraping vLLM metrics (time-to-first-token, tokens/sec, KV cache hit rate, queue depth) into Grafana dashboards with SLO-based alerting.
- Distributed tracing (OpenTelemetry) across gateway → prefill → decode to localize where latency is actually spent.

4. Hardware & memory math

Sizing a model onto GPUs

- Weight memory $\approx$ param count $\times$ bytes per param (2 bytes for fp16/bf16, ~ 1 for int8, ~ 0.5 for int4).
- Example: 70B params $\times$ 2 bytes = 140GB of weights. On 80GB GPUs (A100/H100), that's a minimum of 2 GPUs just to hold weights — $140 \div 80 = 1.75$, round up.
- Real deployments run with more headroom than the bare minimum — typically 4 GPUs for a 70B model — to leave room for KV cache (grows with batch size $\times$ sequence length), activations, and fragmentation.
- Tensor parallelism splits individual layers/matrices across GPUs so they compute in parallel per token — favored for latency since all GPUs work on every token simultaneously (requires fast interconnect, e.g. NVLink).
- Pipeline parallelism splits different layers across different GPUs, passing activations down the pipeline — better tolerates slower interconnect but adds bubble latency and is usually chosen for throughput over latency.

Interview angle: be ready to state the tradeoff plainly — tensor parallelism = lower latency, needs fast GPU-to-GPU links, doesn't scale well past a single node; pipeline parallelism = higher throughput at scale, tolerates slower links, worse latency per request.

5. Cost & optimization levers

- Quantization (int8/int4/fp8) — smaller memory footprint, more requests per GPU, some quality tradeoff to validate via evals.
- Continuous batching — keep GPUs busy by dynamically packing requests into the running batch instead of static batch windows.
- Speculative decoding — a small draft model proposes multiple tokens, the large model verifies them in one pass, cutting decode steps when the draft is accurate.
- Prefix caching / cache-aware routing — avoid recomputing shared system prompts or repeated context across requests.
- Right-sizing hardware to workload — don't run a 70B model when a smaller distilled/fine-tuned model meets the quality bar for the task.

6. Rapid-fire Q&A

Q: Why not just put vLLM instances behind a plain round-robin load balancer?

Round robin ignores state: it doesn't know which instance has a cached prefix (missing free reuse) and doesn't distinguish compute-bound prefill from memory-bound decode, so a big prefill job can stall decode latency on the same instance. An inference-aware gateway routes on cache hits and load instead.

Q: What problem does disaggregated prefill/decode solve?

It prevents prefill's bursty compute demand from stealing GPU cycles from decode, which needs steady access to run smoothly. Separating them into different pools keeps decode latency predictable, at the cost of a KV cache transfer between pools.

Q: Why transfer KV cache directly rather than through shared storage?

Direct GPU-to-GPU transfer (RDMA/NVLink) is low-latency (micro to low milliseconds) and built for the hot path. Shared disk/volume storage has far lower bandwidth and introduces contention under concurrency, making it fine for cache reuse across requests but too slow for the tight prefill-to-decode handoff.

Q: Where does queue depth actually live?

In memory, inside each vLLM instance's own scheduler — exposed as a Prometheus metric. The gateway and autoscaler (KEDA) read it from there rather than from any persisted store, unless an external queue is deliberately placed in front of the pool.

Q: Tensor parallelism vs pipeline parallelism — when would you choose each?

Tensor parallelism splits each layer across GPUs so every GPU works on every token — lowest latency, but needs fast interconnect and doesn't

scale well beyond a node. Pipeline parallelism splits different layers across GPUs in sequence — scales across nodes and tolerates slower links, but adds latency per request and needs careful batching to avoid pipeline bubbles.

Q: How would you size hardware for a 70B model?

Start from weight memory: $\text{params} \times \text{bytes-per-param}$. At fp16 that's 140GB for 70B, which needs at least $2 \times 80\text{GB}$ GPUs just for weights. In practice you add headroom for KV cache and activations and land on around 4 GPUs, using tensor parallelism to split the model.

7. Glossary

PagedAttention — vLLM's memory management technique that stores KV cache in fixed-size blocks (like OS virtual memory pages), eliminating fragmentation and enabling larger batch sizes.

Continuous batching — Dynamically adding and removing requests from an in-flight batch, rather than waiting for a fixed batch window to fill.

KV cache — Cached key/value tensors from prior tokens' attention computation, reused so each new token doesn't require recomputing attention over the full prefix.

Disaggregated serving — Running prefill and decode as separate pools of workers rather than mixed on the same instances.

Tensor parallelism — Splitting individual layers/matrices of a model across multiple GPUs so they compute in parallel per token.

Pipeline parallelism — Splitting different layers of a model across GPUs, with activations passed down the pipeline stage by stage.

Speculative decoding — Using a small draft model to propose several tokens, verified by the large model in a single pass to reduce decode steps.

KEDA — Kubernetes Event-Driven Autoscaling — scales workloads on custom/external metrics rather than just CPU/memory.

Inference Gateway (IGW) — A Kubernetes Gateway API extension for AI-aware, signal-based request routing.

llm-d — A Kubernetes-native distributed inference orchestration stack built on top of vLLM/SGLang.